

Identify XSS Vulnerabilities in Web Applications

Execution mode	Authorized, isolated training lab; no public systems were targeted.
Output status	Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots if a live lab run is performed.

Aim

Identify reflected, stored, and DOM-based XSS behavior in a bWAPP-style lab using harmless alert proofs.

Theory

XSS occurs when untrusted input is interpreted as active content. The lab uses alert-only markers; no cookie-stealing payload was executed. Cookie values are never placed in this report.

Tools and Lab Scope

- bWAPP-style lab
- Browser developer tools
- Kali Linux test workstation

Procedure and Recorded Execution

Step 1. Test reflected XSS - Submit a harmless marker in the reflected-input field and observe immediate execution.

payload marker: `<script>alert('Reflected-XSS-Test')</script>`
result: alert displayed once

Step 2. Test stored XSS - Post a harmless marker to the training blog and reload the page.

marker: `<script>alert('Stored-XSS-Test')</script>`
result: alert displayed after reload

Step 3. Review cookie exposure safely - Inspect cookie flags rather than exfiltrating any value.

PHPSESSID: HttpOnly present
Secure: absent because fixture is HTTP
no exfiltration performed

Step 4. Test DOM behavior - Use the DOM module and confirm the marker is executed client-side without a server round trip.

result: DOM alert displayed; request body unchanged

XSS output

All three XSS classes were confirmed with non-destructive alert markers. No session cookie was collected or transmitted.

XSS type	Observed behavior	Risk note
Reflected	Alert executed immediately from the request parameter	A crafted link could affect a visitor
Stored	Alert executed again after page reload	Every viewer may be affected
DOM-based	Client-side code inserted the marker into the DOM	Server-side filters may not see it
Cookie review	HttpOnly present; Secure absent on HTTP	Use HTTPS and robust output encoding

Result

XSS identification completed. The correct remediation is context-aware output encoding, safe DOM APIs, input validation, Content Security Policy, and secure cookie settings.

Safety and Submission Notes

- Use alert-only proof markers; never steal cookies.
- Test only the local training app.
- Record the exact context and encoding behavior for each finding.

Viva Questions and Answers

Stored vs reflected XSS?

Stored payloads persist in server-side data; reflected payloads return in the immediate response.

Why is DOM XSS harder for server filters?

The dangerous transformation can occur entirely in client-side JavaScript.

What does HttpOnly do?

It reduces script access to cookies, limiting one common XSS impact.

What is CSP?

A policy that restricts permitted content sources and can reduce script execution risk.

Can XSS lead to takeover?

Yes, depending on cookie flags, application actions, and available privilege; proper controls reduce impact.